



Corso di Laurea Magistrale in Informatica

Riproducibilità, analisi comparativa e stabilità dei modelli neurali per la generazione automatica di messaggi di commit

Prof.ssa Filomena Ferrucci
Prof.ssa Sira Vegas
Dott. Gabriele De Vito

Valentina Annunziata
Mat.: 0522501687



Introduzione e Contesto

■	Update README file	45 minutes ago
■	Fixed typo in the documentation that made me slow	2 days ago

🔑	Update README file	1dd25d8
📄	README.md	1 changed file +1 -11
1	1 -This is the public API for FooWidget.	
2	2 Usage	
3	1. Contributing	
4	- +This is the public API for FooWidget.	
5	+ +This is the public API for BarWidget.	

I commit message spiegano *cosa* è cambiato nel codice e *perché*.

Problema: spesso sono: assenti, vaghi (es. “fix”), errati.

La **CMG** (Commit Message Generation) automatizza la scrittura dei commit via AI.

 Come funziona la CMG:
dal codice modificato a un commit message



Mancano confronti sistematici e riproducibili tra modelli.



Identificare modelli riproducibili, pubblici e rappresentativi per il compito di Commit Message Generation (**CMG**).



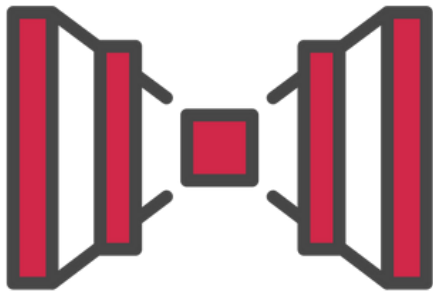
Processo adottato:

- Analisi incrociata di articoli scientifici recenti (2018–2023)
- Verifica disponibilità codice e dataset
- Consultazione con la Prof.ssa Vegas – UPM (Madrid)



Modelli Neurali Analizzati

CoDiSum



RACE



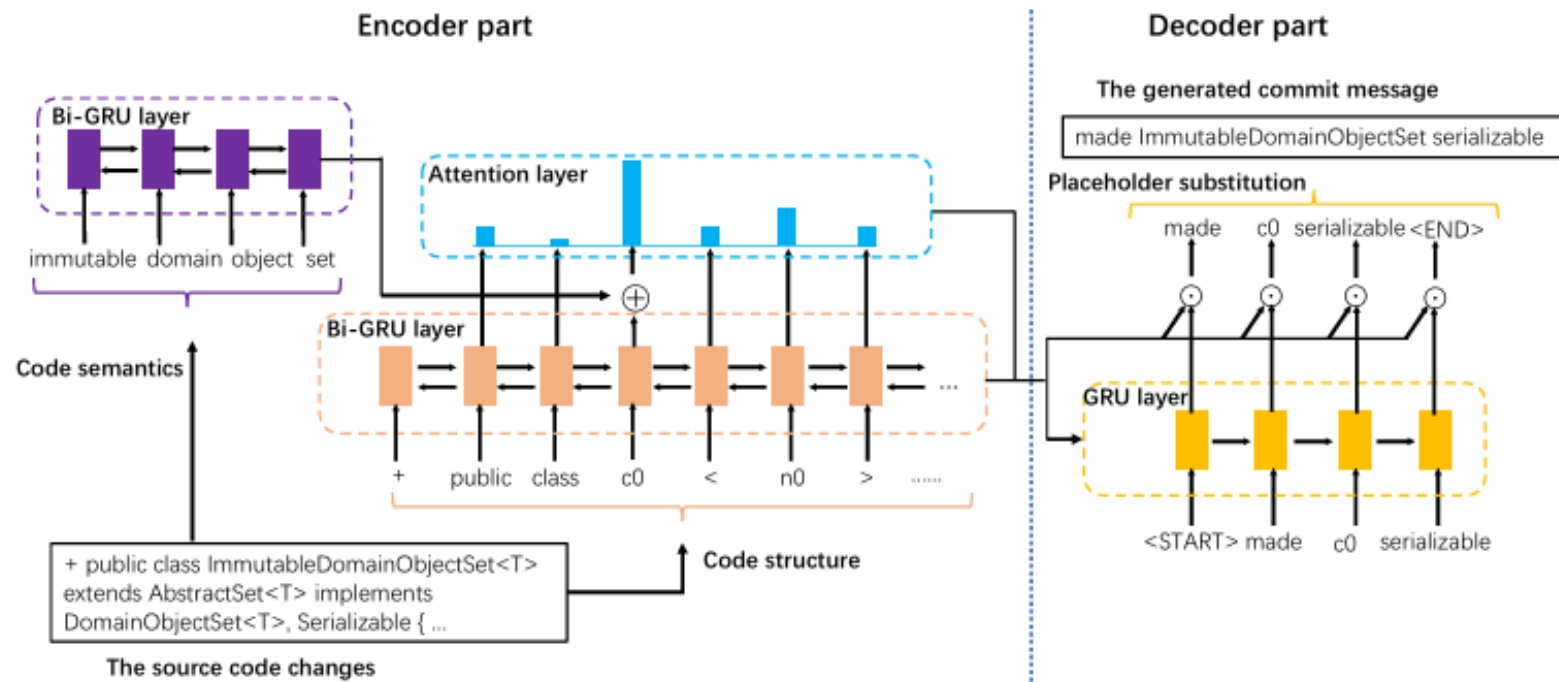
KADEL



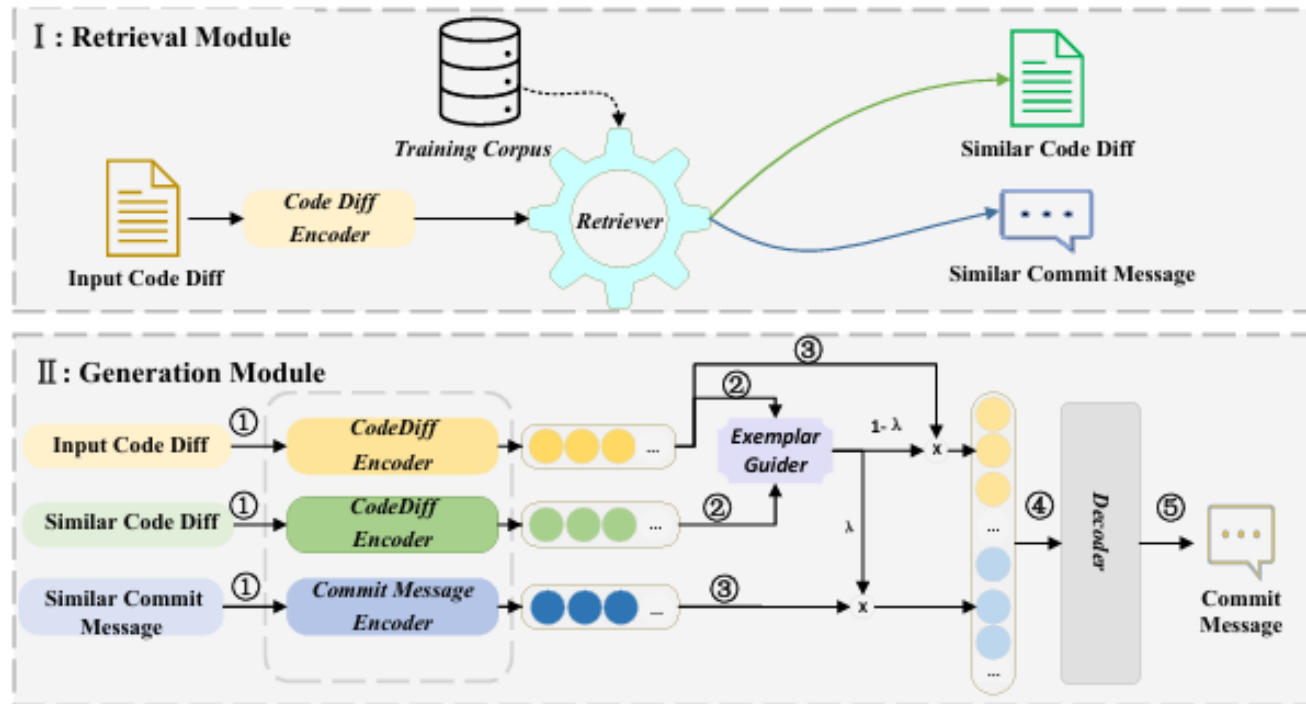
Ogni modello segue un paradigma diverso
confronto su basi architettureali distinte.



CoDiSum

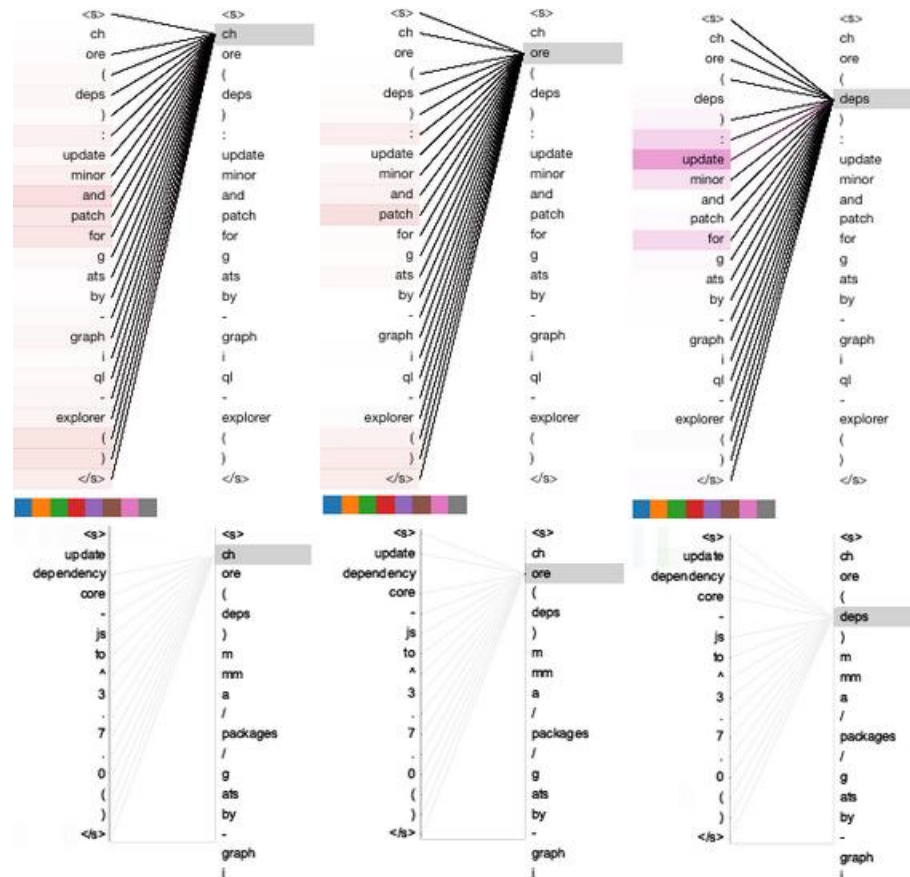


RACE



Selezione dei Modelli

KADEL





RQ1 – Riproducibilità:

- Possiamo ricostruire oggi un modello storico e ottenere risultati coerenti con quelli originali?



RQ2 – Qualità:

- Su un dataset comune, emergono differenze nelle prestazioni tra modelli moderni?



RQ3 – Stabilità:

- Le prestazioni dei modelli rimangono stabili su esecuzioni multiple con inizializzazioni diverse?



Fase 1 - Preparazione dataset

- D1: patch testuali (solo CoDiSum)
- D2: edit actions (RACE, KADEL)
- Pulizia e formattazione coerente



V12 [CoDiSum]

- `train.txt.src / train.txt.tgt,`
- `valid.txt.src / valid.txt.tgt,`
- `test.txt.src / test.txt.tgt.`

90661 coppie (diff, message). split 80/10/10

MCMD [RACE\KADEL]

- `train_small.jsonl,`
- `valid_small.jsonl,`
- `test_small.jsonl.`

50k train, 5k valid/test



Fase 2 - Configurazione tokenizzatore

Modello	Tokenizzatore	Token speciali
CoDiSum	CopyNetPlus (legacy)	<pad>, <unk>, <s>
RACE	CodeT5-base	<INSERT>, <DELETE>, ...
KADEL	CodeT5-small + template	<REPLACE>, <KEEP>, ...

 Troncamento dei diff a **200 token** e dei messaggi a **30 token**



Fase 3 - Costruzione del modello

CoDiSum (CopyNetPlus)

- Architettura GRU
- Attenzione Luong + meccanismo di copia
- Manca la struttura AST e la sostituzione degli identificatori

RACE

- Backbone: CodeT5-base
- Retrieval di un commit simile tramite similarità coseno
- Fusione input + commit esemplare (exemplar guider)
- Beam search

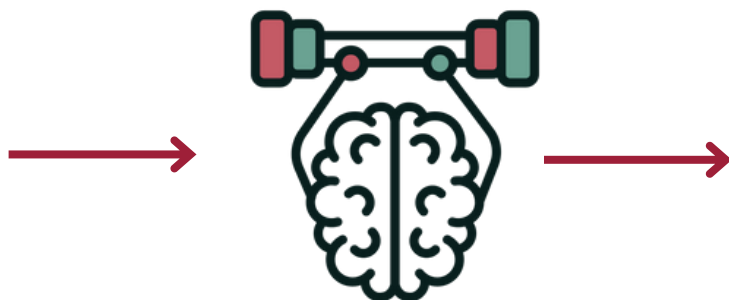
KADEL

- Backbone: CodeT5-small
- Decomposizione del messaggio in componenti “note” e “da generare”
- Co-training: dà più peso agli esempi più affidabili
- Gestione esplicita di segmenti e posizioni



Fase 4 - Addestramento

- GPU Tesla T4 (Colab Pro)
- Seed: 42 / 123 / 999
- Monitoraggio loss durante il training



CoDiSum:

- 10 epoche, no early stopping
- $lr = 1e-4$, batch size 32

RACE:

- 8 epoche, early stopping su BLEU
- $lr = 5e-5$, batch size effettivo 16

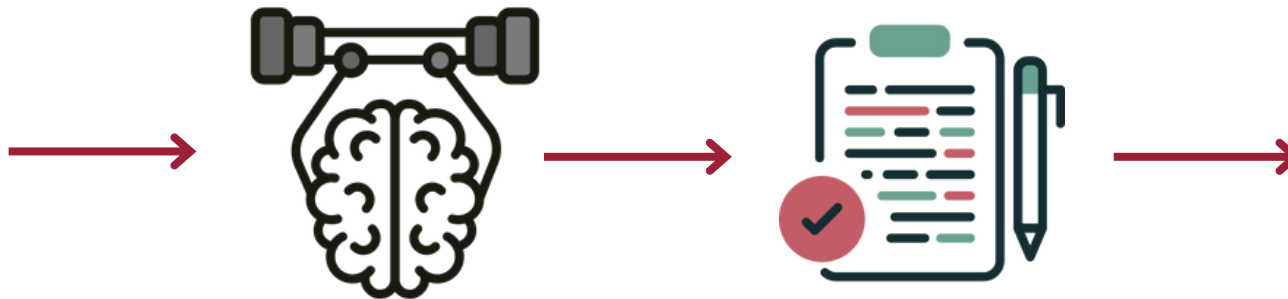
KADEL:

- 10 epoche, early stopping 5 epoche
- $lr = 5e-5$, batch size 8
- co-training + template-aware generation



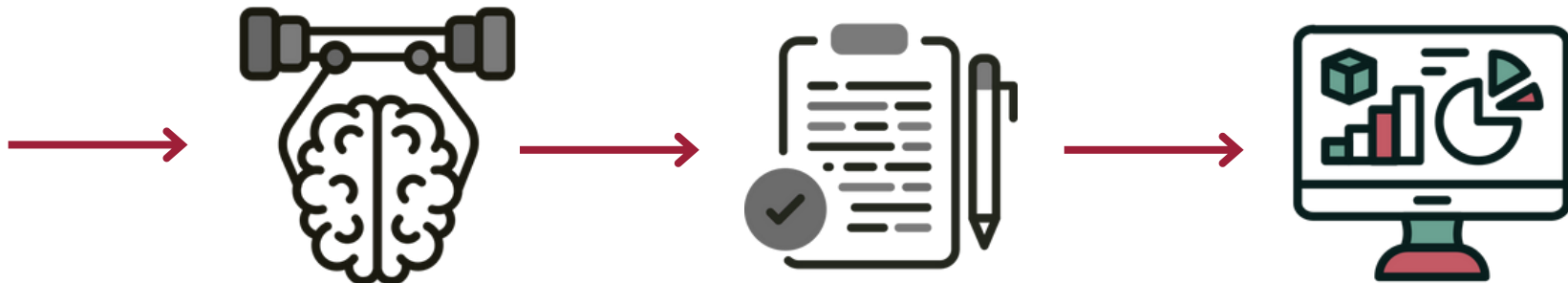
Fase 5 - Valutazione

- Metriche testuali: BLEU, METEOR, ROUGE
- Metriche semantiche: CIDEr, CodeBLEU
- Specificità codice: Identifier Recall

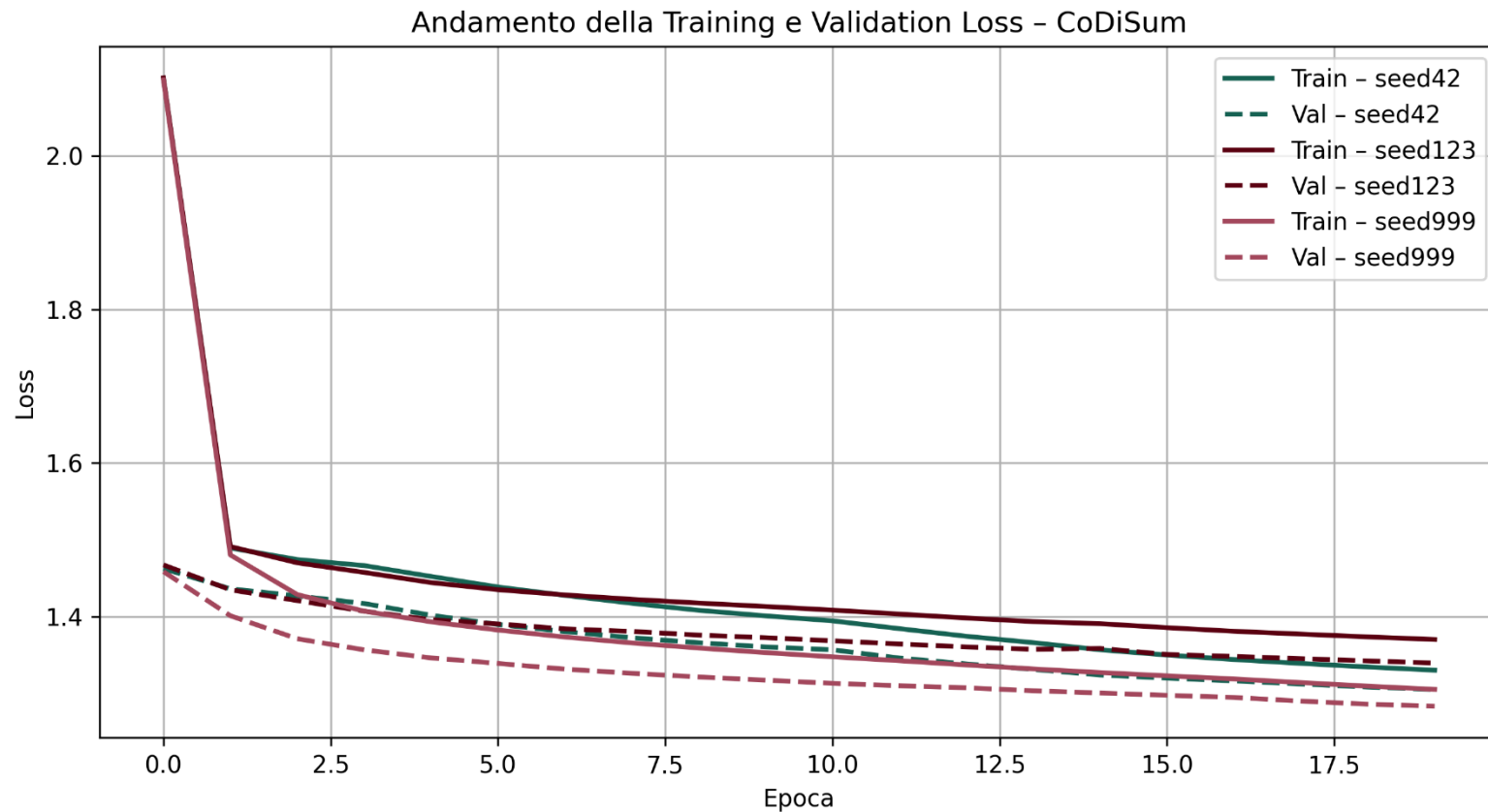


Fase 6 - Analisi statistica

- Shapiro–Wilk → normalità
- Levene → omogeneità varianze
- t-test ($p < 0.05$)
- Cohen's d → ampiezza effetto
- CI al 95%
- Correzione di Holm–Bonferroni



RQ1 – Riproducibilità di CoDiSum



RQ1 – Riproducibilità di CoDiSum

METRICA	RANGE (per seed)
BLEU	0.005-0.007
SacreBLEU	0.06-0.08
METEOR	0.045-0.050
ROUGE-L	0.086-0.091
CodeBLEU-text	0.010-0.016
CIDEr	0.022-0.026
Identifier Recall	0.000 in tutti i casi

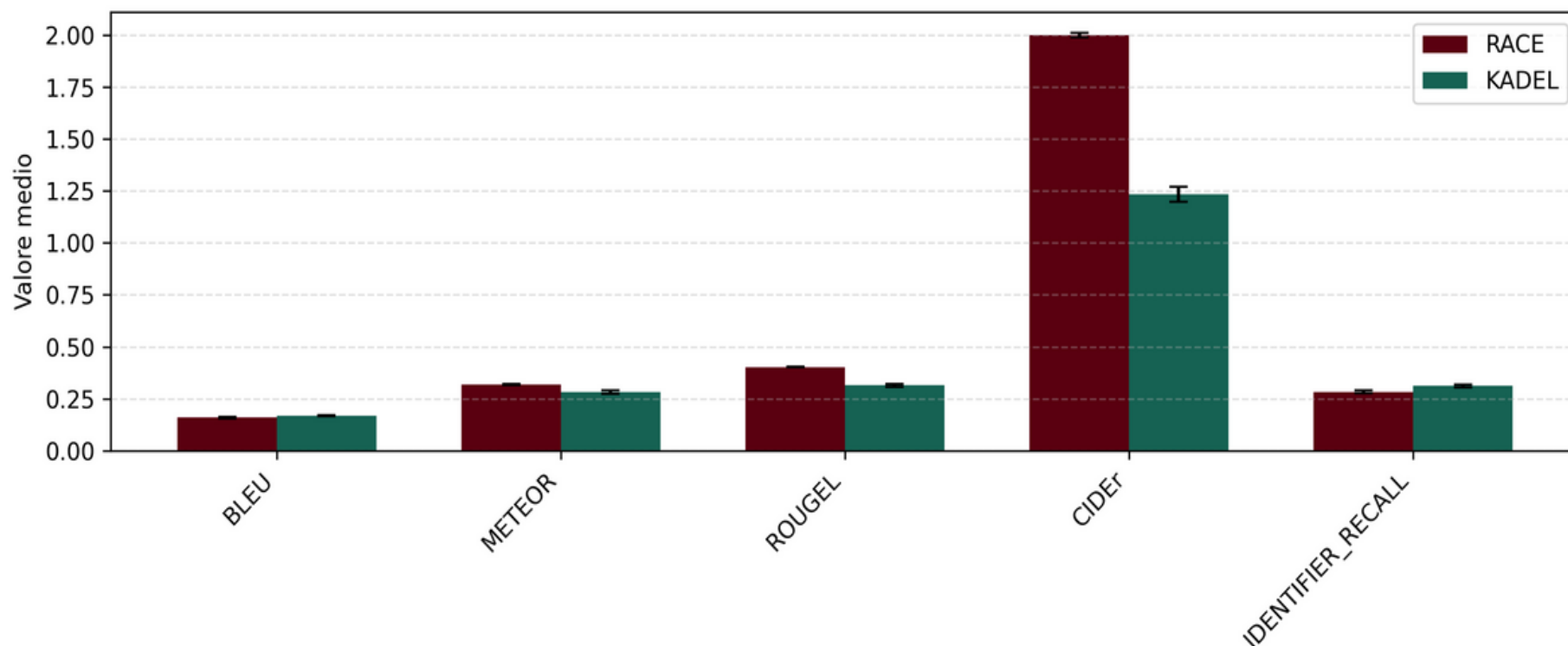
Cause del fallimento

- Artefatto incompleto rispetto al paper (mancano AST, action-level, sostituzione ID).
- Problema indipendente da seed, dataset e pipeline.

Conclusione: CoDiSum non è riproducibile rispetto al paper originale.



RQ2 - Confronto tra RACE e KADEL sul dataset condiviso



RQ2 - Analisi qualitativa e statistica

Metrica	Cohen's d	t-test p-value	95% CI (Δ)
METEOR	5.48	0.0016	[0.025, 0.038]
ROUGE-L	20.63	0.00002	[0.080, 0.092]
CIDEr	57.15	0.00001	[0.74, 0.76]
SacreBLEU	-8.53	0.0003	[-2.67, -1.69]
Identifier Recall	-2.45	0.0445	[-0.036, -0.0005]

RACE è complessivamente il modello più robusto e performante sul dataset condiviso, con vantaggi netti nelle metriche di qualità testuale e semantica.



RQ3 - Analisi della Stabilità

Metrica	CoDiSum σ	RACE σ	KADEL σ
BLEU	≈ 0	0.003	0.002
METEOR	0.003	0.003	0.006
ROUGE-L	0.004	0.002	0.005
CIDEr	0.005	0.009	0.029
Identifier Recall	<i>non applicabile</i> (sempre 0)	0.005	0.007



RQ3 - Analisi della Stabilità

CoDiSum

- Stabilità apparente: le run sono simili ma perché il modello fallisce.
- La bassa variabilità è un effetto del collasso, non una proprietà positiva

RACE

- Deviazioni standard molto basse → comportamento stabile.
- Convergenza consistente tra seed diversi.
- Le metriche sono affidabili anche con solo 3 run.

KADEL

- Variabilità contenuta e comparabile a RACE.
- Il co-training introduce stocasticità, ma l'impatto del seed resta limitato.



Minacce alla Validità



Interna

- Solo 3 run per modello → limita il potere statistico delle analisi.
- CoDiSum incompleto → comportamento degenerato.
- Il gap tra modelli storici e moderni potrebbe essere influenzato dall'artefatto non fedele.



Esterna

- Esperimenti condotti solo sul subset Java → risultati non generalizzabili ad altri linguaggi.
- Ambiente controllato → possibili differenze in contesti industriali reali.

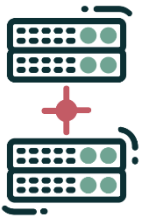




Valutazione umana per testare l'utilità percepita



Test con LLM moderni (CodeT5+, CodeLlama, Codex...)



Dataset migliori: filtraggio + augmentation

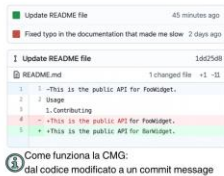


Integrazione di modelli critici per controllo qualità



Introduzione e Contesto

sesa[®]
SOFTWARE ENGINEERING
SALON



I commit message spiegano cosa è cambiato nel codice e perché. Problema: spesso sono: assenti, vaghi (es. "fix"), errati.

La CMG (Commit Message Generation) automatizza la scrittura dei commit via AI.

Come funziona la CMG: dal codice modificato a un commit message

Mancano confronti sistematici e riproducibili tra modelli.

Selezione dei Modelli

sesa[®]
SOFTWARE ENGINEERING
SALON

Modelli Neurali Analizzati

CoDiSum



RACE



KADEL



Ogni modello segue un paradigma diverso → confronto su basi architetturel distinte.

Fasi Sperimentali

sesa[®]
SOFTWARE ENGINEERING
SALON

Fase 6 - Analisi statistica

- Test normalità (Shapiro-Wilk), varianza (Levene)
- t-test o Mann-Whitney
- p-value + Cohen's d + CI 95%



Analisi dei Risultati

sesa[®]
SOFTWARE ENGINEERING
SALON

RQ3 - Analisi della Stabilità

Metrica	CoDiSum σ	RACE σ	KADEL σ
BLEU	≈ 0	354	277
METEOR	339	313	714
ROUGE-L	441	189	654
CIDEr	644	1.056	3.550
Identifier Recall	non applicabile (sempre 0)	640	809

Riproducibilità, analisi comparativa e stabilità dei modelli neurali per la generazione automatica di messaggi di commit

Grazie!



Questa tesi ha contribuito a piantare un albero in Tanzania



Valentina Annunziata

v.annunziata17@studenti.unisa.it

email@studenti.unisa.it

www.linkedin.com/in/v-annunziata

